

NEW WHEELS PROJECT

Context

A lot of people in the world share a common desire: to own a vehicle. A car or an automobile is seen as an object that gives the freedom of mobility. Many now prefer pre-owned vehicles because they come at an affordable cost, but at the same time, they are also concerned about whether the after-sales service provided by the resale vendors is as good as the care you may get from the actual manufacturers. New-Wheels, a vehicle resale company, has launched an app with an end-to-end service from listing the vehicle on the platform to shipping it to the customer's location. This app also captures the overall after-sales feedback given by the customer.

Objective

New-Wheels sales have been dipping steadily in the past year, and due to the critical customer feedback and ratings online, there has been a drop in new customers every quarter, which is concerning to the business. The CEO of the company now wants a quarterly report with all the key metrics sent to him so he can assess the health of the business and make the necessary decisions.

As a data analyst, you see that there is an array of questions that are being asked at the leadership level that need to be answered using data. Import the dump file that contains various tables that are present in the database. Use the data to answer the questions posed and create a quarterly business report for the CEO.

New Wheels Data Dictionary

- shipper_id: Unique ID of the Shipper
- shipper_name: Name of the Shipper
- shipper_contact_details: Contact detail of the Shipper
- product_id: Unique ID of the Product
- vehicle_maker: Vehicle Manufacturing company name
- vehicle_model: Vehicle model name
- vehicle_color: Color of the Vehicle
- vehicle_model_year: Year of Manufacturing
- vehicle_price: Price of the Vehicle
- quantity: Ordered Quantity
- customer_id: Unique ID of the customer
- customer_name: Name of the customer
- gender: Gender of the customer
- job_title: Job Title of the customer
- phone_number: Contact detail of the customer
- email_address: Email address of the customer
- city: Residing city of the customer
- country: Residing country of the customer
- state: Residing state of the customer
- order_date: Date on which customer ordered the vehicle
- order_id: Unique ID of the order
- ship_date: Shipment Date
- ship_mode: Shipping Mode/Class shipping: Shipping Ways
- postal_code: Postal Code of the customer
- discount: Discount given to the customer for the particular order by credit card in percentage
- credit_card_type: Credit Card Type
- credit_card_number: Credit card number
- customer_feedback: Feedback of the customer
- quarter_number : Quarter Number
- customer_address: Address of the customer

Business Question

Question 1

Find the total number of customers who have placed orders. What is the distribution of the customers across states?

Solution Query –

- Total number of customers who placed orders

SELECT

COUNT(DISTINCT customer_id) AS total_customers

FROM order_t;

The screenshot shows a database query execution interface. At the top, there's a toolbar with options like 'Result Grid', 'Filter Rows', 'Exports', and 'Wrap Cell Contents'. Below this, a table displays the query results:

	total_customers
▶	994

On the right side, there's a vertical toolbar with icons for 'Result Grid', 'Form Editor', 'Field Types', and 'Query Stats'. Below the table, there's a tab labeled 'Result 11'. At the bottom, there's an 'Output' section with a dropdown menu set to 'Action Output'. Below this, a log shows the execution details:

#	Time	Action	Message	Duration / Fetch
1	17:48:45	SELECT COUNT(DISTINCT customer_id) AS total_customers FROM order_t LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec

Solution Query – Distribution of customers across states

```
SELECT
    customer_t.state,
    COUNT(DISTINCT order_t.customer_id) AS customer_count
FROM order_t
JOIN customer_t
    ON order_t.customer_id = customer_t.customer_id
GROUP BY customer_t.state
ORDER BY customer_count DESC;
```



state	customer_count
California	97
Texas	97
Florida	86
New York	69
District of Columbia	35
Colorado	33
Ohio	33
Alabama	29
Washington	28
Arizona	26
Illinois	25
Pennsylvania	25
Virginia	24
Missouri	23
Tennessee	23

#	Time	Action	Message	Duration / Fetch
1	17:48:45	SELECT COUNT(DISTINCT customer_id) AS total_customers FROM order_t LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
2	17:53:00	SELECT customer_t.state, COUNT(DISTINCT order_t.customer_id) AS customer_count FROM order_t JOIN customer_t ON o...	49 row(s) returned	0.016 sec / 0.000 sec

Observations and Insights –

- California and Texas found highest no of customers followed by Florida and New York
- Several states like New Mexico, Montana and Maine found lowest customer count like 5,3 and 1
- Launch marketing campaigns in states which found to have lowest customer count.
- Strengthen dealership and service network in high performing states to retain leadership

Question 2

Which are the top 5 vehicle makers preferred by the customers?

Solution Query –

```
SELECT
    product_t.vehicle_maker,
    COUNT(DISTINCT order_t.customer_id) AS customer_count
FROM order_t
JOIN product_t
ON order_t.product_id = product_t.product_id
GROUP BY product_t.vehicle_maker
ORDER BY customer_count DESC
LIMIT 5;
```

Observations and Insights –

- Chevrolet and Ford are most preferred brand
- Demand for Dodge and Pontiac is low
- Maintain higher inventory for Chevrolet and Ford to avoid stockouts
- Run promotional offers for low selling brands to balance portfolio

The screenshot shows a database management tool interface. At the top, there's a toolbar with options like 'Result Grid', 'Filter Rows', 'Export', 'Wrap Cell Contents', and 'Fetch rows'. Below this is a table with two columns: 'vehicle_maker' and 'customer_count'. The table contains five rows of data. To the right of the table is a vertical sidebar with icons for 'Result Grid', 'Form Editor', 'Field Types', and 'Query Stats'. Below the table, there's a section labeled 'Result 13' with a close button. Underneath that is an 'Output' section with a dropdown menu set to 'Action Output'. The output section contains a log of two database actions, each with a green checkmark icon, a timestamp, the SQL query, the message, and the duration/fetch time.

	vehicle_maker	customer_count
▶	Chevrolet	83
	Ford	63
	Toyota	52
	Dodge	50
	Pontiac	50

#	Time	Action	Message	Duration / Fetch
2	17:53:00	SELECT customer_t.state, COUNT(DISTINCT order_t.customer_id) AS customer_count FROM order_t JOIN customer_t ON...	49 row(s) returned	0.016 sec / 0.000 sec
3	17:59:00	SELECT product_t.vehicle_maker, COUNT(DISTINCT order_t.customer_id) AS customer_count FROM order_t JOIN product_t...	5 row(s) returned	0.015 sec / 0.000 sec

Question 3

Which is the most preferred vehicle maker in each state?

Solution Query –

```
SELECT
    customer_t.state,
    product_t.vehicle_maker,
    COUNT(DISTINCT order_t.customer_id) AS customer_count
FROM order_t
JOIN customer_t
    ON order_t.customer_id = customer_t.customer_id
JOIN product_t
    ON order_t.product_id = product_t.product_id
GROUP BY customer_t.state, product_t.vehicle_maker;
```

Observations and Insights –

- Customer preference varies significantly by state
- No single maker dominates all regions
- Local culture and service availability influence brand choice
- Partner with local service provider to dominate in each state.

The screenshot displays a data management tool interface. The top section, titled 'Result Grid', shows a table with three columns: 'state', 'vehicle_maker', and 'customer_count'. The data is filtered for the state of Alabama, listing 15 different vehicle makers and their corresponding customer counts. The bottom section, titled 'Output', shows a log of actions performed. It includes a table with columns for '#', 'Time', 'Action', 'Message', and 'Duration / Fetch'. Two actions are listed: a SELECT query for product_t.vehicle_maker and a SELECT query for customer_t.state, both returning 524 rows.

state	vehicle_maker	customer_count
Alabama	Buick	1
Alabama	Chevrolet	2
Alabama	Dodge	5
Alabama	Ford	3
Alabama	Honda	1
Alabama	Hyundai	1
Alabama	Infiniti	1
Alabama	Land Rover	1
Alabama	Lexus	1
Alabama	Lincoln	1
Alabama	Mazda	2
Alabama	Mitsubishi	1
Alabama	Morgan	1
Alabama	Oldsmobile	1
Alabama	Plymouth	2

#	Time	Action	Message	Duration / Fetch
3	17:59:00	SELECT product_t.vehicle_maker, COUNT(DISTINCT order_t.customer_id) AS customer_count FROM order_t JOIN product_t...	5 row(s) returned	0.015 sec / 0.000 sec
4	18:03:21	SELECT customer_t.state, product_t.vehicle_maker, COUNT(DISTINCT order_t.customer_id) AS customer_count FROM or...	524 row(s) returned	0.016 sec / 0.000 sec

Question 4

Find the overall average rating given by the customers. What is the average rating in each quarter?

Solution Query –

```
SELECT  
AVG(  
  CASE customer_feedback  
    WHEN 'Very Bad' THEN 1  
    WHEN 'Bad' THEN 2  
    WHEN 'Okay' THEN 3  
    WHEN 'Good' THEN 4  
    WHEN 'Very Good' THEN 5  
  ) AS overall_avg_rating  
FROM order_t;
```



The screenshot shows a database query execution interface. At the top, there's a 'Result Grid' with a single row containing the value '3.1350' under the column 'overall_avg_rating'. Below this, there's an 'Output' section with a table showing the execution details of the query. The table has columns for 'Time', 'Action', 'Message', and 'Duration / Fetch'. The first row shows the query execution at 18:03:21, returning 624 rows. The second row shows the query execution at 18:49:52, returning 1 row.

Time	Action	Message	Duration / Fetch
4 18:03:21	SELECT customer_t.state, product_t.vehicle_maker, COUNT(DISTINCT order_t.customer_id) AS customer_count FROM ar...	624 row(s) returned	0.016 sec / 0.000 sec
5 18:49:52	SELECT AVG(CASE customer_feedback WHEN 'Very Bad' THEN 1 WHEN 'Bad' THEN 2 WHEN 'Okay' THEN 3 WHEN 'Good' THEN 4 WHEN 'Very Good' THEN 5) AS overall_avg_rating FROM order_t;	1 row(s) returned	0.110 sec / 0.000 sec

Solution Query –

```
SELECT
    quarter_number,
    AVG(rating_value) AS avg_rating
FROM
(
    SELECT
        quarter_number,

        CASE customer_feedback
            WHEN 'Very Bad' THEN 1
            WHEN 'Bad' THEN 2
            WHEN 'Okay' THEN 3
            WHEN 'Good' THEN 4
            WHEN 'Very Good' THEN 5
        END AS rating_value

    FROM order_t
) AS feedback_table

GROUP BY quarter_number
ORDER BY quarter_number;
```

Observations and Insights –

- Overall rating is around the Good range
- Quarter 3 and 4 show decline in avg rating
- Investigate low quarters to identify root cause.

The screenshot shows a data tool interface. At the top, there's a 'Result Grid' tab with a table of quarterly average ratings. Below it, the 'Output' section shows a log of actions, including a SQL query that calculates the average rating by quarter.

quarter_number	avg_rating
1	3.5548
2	3.3550
3	2.9563
4	2.3970

#	Time	Action	Message	Duration / Fetch
5	18:49:52	SELECT AVG(CASE customer_feedback WHEN 'Very Bad' THEN 1 WHEN 'Bad' THEN 2 WHEN 'Okay' THEN ...	1 row(s) returned	0.110 sec / 0.000 sec
6	18:51:57	SELECT quarter_number, AVG(rating_value) AS avg_rating FROM (SELECT quarter_number, CASE customer_f...	4 row(s) returned	0.032 sec / 0.000 sec

Question 5

Find the percentage distribution of feedback from the customers. Are customers getting more dissatisfied over time?

Solution Query –

```
SELECT
    quarter_number,
    ROUND(
        SUM(CASE WHEN customer_feedback = 'Very Good' THEN 1 ELSE 0 END)
        * 100.0 / COUNT(*), 2
    ) AS very_good_percent,

    ROUND(
        SUM(CASE WHEN customer_feedback = 'Good' THEN 1 ELSE 0 END)
        * 100.0 / COUNT(*), 2
    ) AS good_percent,

    ROUND(
        SUM(CASE WHEN customer_feedback = 'Okay' THEN 1 ELSE 0 END)
        * 100.0 / COUNT(*), 2
    ) AS okay_percent,
```

```
ROUND(  
    SUM(CASE WHEN customer_feedback = 'Bad' THEN 1 ELSE 0 END)  
    * 100.0 / COUNT(*), 2  
) AS bad_percent,
```

```
ROUND(  
    SUM(CASE WHEN customer_feedback = 'Very Bad' THEN 1 ELSE 0 END)  
    * 100.0 / COUNT(*), 2  
) AS very_bad_percent
```

```
FROM order_t  
GROUP BY quarter_number  
ORDER BY quarter_number;
```

Observations and Insights –

- If bad and Very bad percent rises then the dissatisfaction rises
- Experience consistency is a challenge
- Create early warning dashboard for negative feedback
- Train sales and delivery teams on customer experience

The screenshot displays a software interface for data management. At the top, there's a toolbar with options like 'Result Grid', 'Filter Rows', 'Exports', and 'Wrap Cell Contents'. Below this is a table with 7 columns: 'quarter_number', 'very_good_percent', 'good_percent', 'okay_percent', 'bad_percent', and 'very_bad_percent'. The table contains 4 rows of data. To the right of the table is a vertical sidebar with icons for 'Result Grid', 'Form Editor', 'Field Types', and 'Query Stats'. Below the table, there's a section labeled 'Output' with a dropdown menu set to 'Action Output'. This section shows a log of database actions with columns for '#', 'Time', 'Action', 'Message', and 'Duration / Fetch'. Two actions are listed, both successful.

quarter_number	very_good_percent	good_percent	okay_percent	bad_percent	very_bad_percent
1	30.00	28.71	19.03	11.29	10.97
2	28.63	22.14	20.23	14.12	14.89
3	16.59	20.96	21.83	22.71	17.90
4	10.05	10.05	20.10	29.15	30.65

#	Time	Action	Message	Duration / Fetch
6	18:51:57	SELECT quarter_number, AVG(rating_value) AS avg_rating FROM (SELECT quarter_number, CASE customer_f...	4 row(s) returned	0.032 sec / 0.000 sec
7	10:34:30	SELECT quarter_number, ROUND(SUM(CASE WHEN customer_feedback = 'Very Good' THEN 1 ELSE 0 END) * ...	4 row(s) returned	0.109 sec / 0.000 sec

Question 6

What is the trend of the number of orders by quarter?

Solution query –

SELECT

quarter_number,

COUNT(order_id) AS total_orders

FROM order_t

GROUP BY quarter_number

ORDER BY quarter_number;



	quarter_number	total_orders
1	1	310
2	2	262
3	3	229
4	4	199

#	Time	Action	Message	Duration / Fetch
7	10:34:30	SELECT quarter_number, ROUND(SUM(CASE WHEN customer_feedback = "Very Good" THEN 1 ELSE 0 END))	4 row(s) returned	0.109 sec / 0.000 sec
8	10:56:55	SELECT quarter_number, COUNT(order_id) AS total_orders FROM order_t GROUP BY quarter_number ORDER BY quarter_number	4 row(s) returned	0.016 sec / 0.000 sec

Observations and Insights –

- Order volume shows seasonal fluctuations
- Some quarter contribute significantly higher sales
- Demand is not evenly distributed
- Align marketing calendar with high demand periods

Question 7

Calculate the net revenue generated by the company. What is the quarter-over-quarter % change in net revenue?

Solution query –

```
SELECT
    quarter_number,

    SUM(    order_t.quantity * product_t.vehicle_price *
    (1 - order_t.discount / 100)
    ) AS net_revenue,

    LAG(
        SUM(order_t.quantity * product_t.vehicle_price *
        (1 - order_t.discount / 100)
        )
    ) OVER (ORDER BY quarter_number) AS previous_quarter_revenue,

    ROUND(
    (
        SUM(order_t.quantity * product_t.vehicle_price *
        (1 - order_t.discount / 100)
        )
```

```
LAG(  
    SUM(order_t.quantity * product_t.vehicle_price *  
        (1 - order_t.discount / 100)  
    )  
    ) OVER (ORDER BY quarter_number)  
)  
/  
LAG(  
    SUM(order_t.quantity * product_t.vehicle_price *  
        (1 - order_t.discount / 100)  
    )  
    ) OVER (ORDER BY quarter_number)  
    *100 , 2  
)AS qoq_percent_change
```

```
FROM order_t  
JOIN product_t  
    ON order_t.product_id = product_t.product_id  
GROUP BY quarter_number  
ORDER BY quarter_number;
```

Observations and Insights –

- Negative QoQ change indicates margin pressure
- Discounts heavily impact profitability
- Promote high margin models in declining quarters
- Revenue seen reduced while moving to next quarter

The screenshot displays a data analytics interface. At the top, there's a 'Result Grid' tab with options for 'Filter Rows', 'Export', and 'Wrap Cell Content'. Below this is a table with four columns: 'quarter_number', 'net_revenue', 'previous_quarter_revenue', and 'qoq_percent_change'. The table contains four rows of data, with the first row highlighted. To the right of the table is a vertical sidebar with icons for 'Result Grid', 'Form Editor', 'Field Types', and 'Query Stats'. Below the table, there's a 'Result 19' tab and a 'Read Only' indicator. At the bottom, there's an 'Output' section with a dropdown menu set to 'Action Output'. This section contains a log of database actions, including two SELECT queries with their execution times and messages.

quarter_number	net_revenue	previous_quarter_revenue	qoq_percent_change
1	39421580.15929600000000	39421580.15929600000000	
2	32715830.39237633800000	39421580.15929600000000	-17.01
3	29229896.19364900000000	32715830.39237633800000	-10.66
4	23346779.51435220000000	29229896.19364900000000	-20.13

#	Time	Action	Message	Duration / Fetch
8	10:56:55	SELECT quarter_number, COUNT(order_id) AS total_orders FROM order_t GROUP BY quarter_number ORDER BY quarter_n...	4 row(s) returned	0.016 sec / 0.000 sec
9	11:00:29	SELECT quarter_number, SUM(order_t.quantity * product_t.vehicle_price * (1 - order_t.discount / 100)) AS net_...	4 row(s) returned	0.031 sec / 0.000 sec

Question 8

What is the trend of net revenue and orders by quarters?

Solution Query –

```
SELECT
    order_t.quarter_number,

    COUNT(order_t.order_id) AS total_orders,

    SUM(
        order_t.quantity * product_t.vehicle_price *
        (1 - order_t.discount / 100)
    ) AS net_revenue

FROM order_t
JOIN product_t
    ON order_t.product_id = product_t.product_id
GROUP BY order_t.quarter_number
ORDER BY order_t.quarter_number;
```

Observations and Insights –

- Track revenue per customer metric
- Encourage upselling of premium variants

Result Grid

Filter Rows:

Exports

Wrap Cell Contents

	quarter_number	total_orders	net_revenue
▶	1	310	39421580.15929600000000
	2	262	32715830.39237633800000
	3	229	29229896.19364900000000
	4	199	23346779.51435220000000

Result 20

Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
9	11:00:29	SELECT quarter_number, SUM(order_t.quantity * product_t.vehicle_price * (1 - order_t.discount / 100)) AS net_...	4 row(s) returned	0.031 sec / 0.000 sec
10	19:49:02	SELECT order_t.quarter_number, COUNT(order_t.order_id) AS total_orders, SUM(order_t.quantity * product_t.vehicle...	4 row(s) returned	0.109 sec / 0.000 sec

Question 9

What is the average discount offered for different types of credit cards?

Solution query –

```
SELECT
    customer_t.credit_card_type,
    AVG(order_t.discount) AS avg_discount
FROM order_t
JOIN customer_t
    ON order_t.customer_id = customer_t.customer_id
GROUP BY customer_t.credit_card_type
ORDER BY avg_discount DESC;
```


Observations and Insights –

- The avg discounts for all the credit cards lies between 0.58 – 0.64 range
- Unequal discounts may affect profitability
- Standardize discounts to avoid bias

The screenshot displays a database management interface. The top section, titled 'Result Grid', shows a table with two columns: 'credit_card_type' and 'avg_discount'. The table lists 15 credit card types and their corresponding average discount values. The bottom section, titled 'Output', shows a log of database actions. The log includes columns for '#', 'Time', 'Action', 'Message', and 'Duration / Fetch'. Two actions are visible: a SELECT query at 20:00:34 and another at 20:01:34, both returning 16 rows.

credit_card_type	avg_discount
laser	0.643846
mastercard	0.629500
maestro	0.624219
visa-electron	0.623469
china-unionpay	0.622174
instapayment	0.620625
americanexpress	0.616327
diners-club-us-ca	0.614615
diners-club-carte-blanche	0.614490
switch	0.610233
bankcard	0.609545
jcb	0.607382
visa	0.600833
diners-club-enroute	0.599792
solo	0.585000

#	Time	Action	Message	Duration / Fetch
13	20:00:34	SELECT	customer_t.credit_card_type, AVG(order_t.discount) AS avg_discount FROM order_t JOIN customer_t ON order_t....	16 row(s) returned 0.047 sec / 0.000 sec
14	20:01:34	SELECT	customer_t.credit_card_type, AVG(order_t.discount) AS avg_discount FROM order_t JOIN customer_t ON order_t....	16 row(s) returned 0.015 sec / 0.000 sec

Question 10

What is the average time taken to ship the placed orders for each quarter?

Solution query –

```
SELECT
    quarter_number,
    AVG(
        DATEDIFF(ship_date, order_date)
    ) AS avg_shipping_days
FROM order_t
GROUP BY quarter_number
ORDER BY quarter_number;
```

Observations and Insights –

- Avg shipping days seen to increased while moving to next quarter
- Shipping days should be constant with each quarter
- Higher delivery time may lead to poor feedback

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	quarter_number	avg_shipping_days
▶	1	57.1677
	2	71.1107
	3	117.7555
	4	174.0955

Result 24 x

Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✖	16 20:05:42	SELECT quarter_number, AVG(julianday(ship_date) - julianday(order_date)) AS avg_shipping_days FROM order_t G...	Error Code: 1305. FUNCTION new_wheels.julianday does not exist	0.141 sec
✔	17 20:07:10	SELECT quarter_number, AVG(DATEDIFF(ship_date, order_date)) AS avg_shipping_days FROM order_t GROUP B...	4 row(s) returned	0.031 sec / 0.000 sec